Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 14, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your own IBM Cloud project. That is where the learning happens.

Volume 14 — Assessment & Answer Guide

মূল্যায়ন নির্দেশিকা — পরিচিতি ও ব্যবহারবিধি

এই নির্দেশিকা Volume 14-এর ২৪টি অধ্যায়ের শেখা যাচাই করার জন্য তৈরি। এটি মুখস্থবিদ্যার পরীক্ষা নয় — প্রতিটি প্রশ্ন, পরিস্থিতি ও কেস স্টাডি বাস্তব production সিদ্ধান্ত, architecture trade-off এবং operational reasoning যাচাই করে। TechNova একটি কাল্পনিক কোম্পানি; সব উদাহরণ, IP (203.0.113.x), ডোমেইন (technova.example) ও credential (CHANGE-ME-...) কাল্পনিক।

কঠিনতার স্তর

- **Beginner:** মৌলিক ধারণা ও সঠিক ব্যবহার চেনা। অধ্যায় পড়ে থাকলে উত্তর দেওয়া যায়।
- **Intermediate:** দুটি ভালো বিকল্পের মধ্যে প্রেক্ষাপট অনুযায়ী সঠিকটি বেছে নেওয়া; কনফিগারেশন ও ট্রেড-অফ বোঝা।
- **Advanced:** ব্যর্থতা, খরচ, নিরাপত্তা ও multi-cloud জটিলতা একসঙ্গে বিবেচনা করে সিদ্ধান্ত নেওয়া।

কীভাবে ব্যবহার করবেন

- প্রথমে উত্তর না দেখে MCQ সমাধান করুন; তারপর **answer-guide** ও **answer-guide-2**-এর ব্যাখ্যা পড়ুন। ভুল উত্তরের ব্যাখ্যাটাই আসল শেখা।
- Short-answer প্রশ্নের উত্তর ৩–৫ বাক্যে, নিজের ভাষায় লিখুন; তারপর নির্দেশিকার উত্তরের সঙ্গে মূল পয়েন্ট মেলান।
- Scenario ও case study একা বা দলে আলোচনা করে সমাধান করুন — “প্রথমে কী চেক করবেন এবং কেন” অংশটিই মূল্যায়নের কেন্দ্র।
- Final exam নিজেকে ৩ ঘণ্টা সময় দিয়ে, নোট বন্ধ রেখে দিন; তারপর marking guidance দিয়ে নিজেকে নম্বর দিন।
- Competency sheet প্রতি অধ্যায় শেষে পূরণ করুন; দুর্বল জায়গায় ফিরে গিয়ে সংশ্লিষ্ট অধ্যায় ও quick reference পড়ুন।
- কোনো সেবার দাম, limit বা capability এই বই থেকে মুখস্থ করবেন না — production সিদ্ধান্তের আগে সবসময় official documentation যাচাই করুন। IBM Cloud Code Engine বর্তমান serverless প্ল্যাটফর্ম; Cloud Functions legacy হিসেবে বিবেচনা করুন।

নম্বর ও পাসের ধারণা

MCQ প্রতিটি ১ নম্বর, short-answer প্রতিটি ৩ নম্বর, scenario প্রতিটি ৫ নম্বর ধরে নিজেকে যাচাই করতে পারেন। ৭০% বা তার বেশি হলে সংশ্লিষ্ট অধ্যায়গুলোতে আপনি production-প্রস্তুত ভিত্তি পেয়েছেন বলে ধরা যায়; ৭০%-এর নিচে হলে ভুল প্রশ্নগুলোর অধ্যায়ে ফিরে যান।

Volume 14 — Assessment & Answer Guide

বহুনির্বাচনী প্রশ্ন (MCQ) — ১২০টি

প্রতিটি প্রশ্নে একটি সঠিক উত্তর আছে। উত্তর এই অংশে নেই — দেখুন **answer-guide** ও **answer-guide-2**।

1. **প্রশ্ন ১ (Chapter ১ — Beginner):** TechNova তাদের monolith অ্যাপ ক্লাউডে নিতে চায়। প্রথম সিদ্ধান্ত হিসেবে কোনটি সবচেয়ে যুক্তিযুক্ত?
    1. সরাসরি সবচেয়ে বড় virtual server নেওয়া
    2. workload-এর ধরন, scaling প্রয়োজন ও ডেটা অবস্থান চিহ্নিত করা
    3. সব সার্ভিস একই region-এ না দেখে বেছে নেওয়া
    4. প্রথমে billing অ্যাকাউন্ট বন্ধ করা
2. **প্রশ্ন ২ (Chapter ১ — Intermediate):** Public, private ও hybrid cloud-এর মধ্যে বেছে নেওয়ার সময় প্রধান বিবেচ্য কোনটি?
    1. শুধু vendor-এর ব্র্যান্ড
    2. ডেটা সংবেদনশীলতা, compliance, latency ও খরচ একসঙ্গে
    3. শুধু ড্যাশবোর্ডের রং
    4. শুধু কর্মীসংখ্যা
3. **প্রশ্ন ৩ (Chapter ১ — Intermediate):** Cloud-এ “shared responsibility model” বলতে কী বোঝায়?
    1. সব নিরাপত্তা provider-এর দায়িত্ব
    2. সব নিরাপত্তা গ্রাহকের দায়িত্ব
    3. provider ও গ্রাহকের মধ্যে স্তর অনুযায়ী দায়িত্ব ভাগ হয়
    4. কোনো পক্ষেরই দায়িত্ব নেই
4. **প্রশ্ন ৪ (Chapter ১ — Advanced):** TechNova-র একটি workload-এ হঠাৎ traffic ১০ গুণ বাড়ে, আবার কমে যায়। কোন architecture সবচেয়ে উপযুক্ত?
    1. একটি fixed বড় সার্ভার
    2. auto-scaling সহ stateless design ও managed সার্ভিস
    3. সব ডেটা একটিমাত্র ডিস্কে রাখা
    4. manual deploy প্রতিদিন
5. **প্রশ্ন ৫ (Chapter ১ — Beginner):** IaaS, PaaS ও SaaS-এর পার্থক্য মূলত কী নিয়ে?
    1. শুধু দাম
    2. গ্রাহক কতটুকু infrastructure নিজে পরিচালনা করে
    3. শুধু region সংখ্যা
    4. শুধু প্রোগ্রামিং ভাষা

6. **প্রশ্ন ৬ (Chapter ২ — Beginner):** IBM Cloud অ্যাকাউন্টে resource group-এর মূল উদ্দেশ্য কী?
    1. শুধু নাম সুন্দর করা
    2. resource সংগঠিত করা, access ও billing আলাদা করা
    3. সার্ভারের গতি বাড়ানো
    4. ডেটা মুছে ফেলা
7. **প্রশ্ন ৭ (Chapter ২ — Intermediate):** TechNova-র dev, staging ও prod আলাদা রাখতে চায়। সেরা পদ্ধতি কোনটি?
    1. একই resource group-এ সব রাখা
    2. পরিবেশভিত্তিক আলাদা resource group ও আলাদা access policy
    3. একই API key সব পরিবেশে ব্যবহার
    4. prod-এর credential dev-এ কপি করা
8. **প্রশ্ন ৮ (Chapter ২ — Intermediate):** IBM Cloud CLI-তে login করার পর কোন কমান্ডে target resource group যাচাই করবেন?
    1. ibmcloud target
    2. ibmcloud login --help
    3. ibmcloud billing
    4. ibmcloud logout
9. **প্রশ্ন ৯ (Chapter ২ — Advanced):** একাধিক টিম একই অ্যাকাউন্ট ব্যবহার করলে খরচের দায় কার তা জানার সেরা উপায় কী?
    1. অনুমান করা
    2. resource group ও tag দিয়ে cost allocation করা
    3. সব resource এক নামে রাখা
    4. billing বন্ধ রাখা
10. **প্রশ্ন ১০ (Chapter ২ — Beginner):** Region বেছে নেওয়ার সময় কোনটি বিবেচনা করবেন না?
    1. ব্যবহারকারীর অবস্থান ও latency
    2. সার্ভিসের উপলভ্যতা
    3. ডেটা residency প্রয়োজন
    4. ড্যাশবোর্ডের থিম
11. **প্রশ্ন ১১ (Chapter ৩ — Beginner):** IAM-এ least privilege নীতির অর্থ কী?
    1. সবাইকে admin দেওয়া
    2. কাজের জন্য যতটুকু দরকার ঠিক ততটুকু access দেওয়া
    3. কাউকে access না দেওয়া
    4. password শেয়ার করা

12. **প্রশ্ন ১২ (Chapter ৩ — Intermediate):** একজন developer-কে শুধু একটি resource group-এর VPC দেখতে দিতে চাইলে কী করবেন?
    1. Account admin করে দেবেন
    2. ওই resource group-এ সীমিত Viewer role-এর policy দেবেন
    3. root API key শেয়ার করবেন
    4. সব group-এ Editor দেবেন
13. **প্রশ্ন ১৩ (Chapter ৩ — Intermediate):** Service ID কখন ব্যবহার করা উচিত?
    1. ব্যক্তিগত login-এর বদলে, অ্যাপ বা automation-এর পরিচয় হিসেবে
    2. পাসওয়ার্ড মনে রাখতে
    3. সব মানুষের জন্য একই login হিসেবে
    4. billing দেখতে
14. **প্রশ্ন ১৪ (Chapter ৩ — Advanced):** একটি সার্ভিসকে অন্য সার্ভিসের ডেটা পড়তে দিতে হবে, কিন্তু মানুষের credential দেওয়া যাবে না। সঠিক পদ্ধতি কোনটি?
    1. ব্যক্তিগত API key কোডে বসানো
    2. service-to-service authorization ও সীমিত role ব্যবহার
    3. public করে দেওয়া
    4. password ইমেইলে পাঠানো
15. **প্রশ্ন ১৫ (Chapter ৩ — Beginner):** API key ফাঁস হলে প্রথম করণীয় কী?
    1. অপেক্ষা করা
    2. key revoke/rotate করা ও ব্যবহার নিরীক্ষা করা
    3. key আরও জায়গায় বসানো
    4. কাউকে না জানানো
16. **প্রশ্ন ১৬ (Chapter ৪ — Beginner):** VPC-তে subnet ভাগ করার প্রধান উদ্দেশ্য কী?
    1. IP নষ্ট করা
    2. network আলাদা করা, নিরাপত্তা ও routing নিয়ন্ত্রণ
    3. সার্ভার বন্ধ করা
    4. খরচ লুকানো
17. **প্রশ্ন ১৭ (Chapter ৪ — Intermediate):** Public subnet-এর workload-কে ইন্টারনেটে যেতে দিতে, কিন্তু বাইরে থেকে ঢুকতে না দিতে কী ব্যবহার করবেন?
    1. Public Gateway / NAT ধরনের egress ব্যবস্থা
    2. সব port খুলে দেওয়া
    3. Security Group মুছে ফেলা
    4. public IP সব instance-এ দেওয়া

18. **প্রশ্ন ১৮ (Chapter ৪ — Intermediate):** Security Group ও NACL-এর মূল পার্থক্য কী?
    1. কোনো পার্থক্য নেই
    2. Security Group stateful ও instance-স্তরে; NACL stateless ও subnet-স্তরে
    3. NACL শুধু billing করে
    4. Security Group শুধু DNS করে

19. **প্রশ্ন ১৯ (Chapter ৪ — Advanced):** TechNova-র on-premises ডেটা সেন্টার ও IBM Cloud VPC-এর মধ্যে স্থিতিশীল, ব্যক্তিগত সংযোগ দরকার। সেরা বিকল্প কোনটি?
    1. public ইন্টারনেটে খোলা port
    2. Direct Link ধরনের dedicated/private সংযোগ
    3. password শেয়ার করা VPN ছাড়া
    4. ডেটা USB-তে পাঠানো

20. **প্রশ্ন ২০ (Chapter ৪ — Beginner):** 10.0.0.0/24 CIDR-এ কতটি ব্যবহারযোগ্য host ঠিকানা সাধারণত ধরা হয়?
    1. প্রায় ২৫৬টি ঠিকানা, কিছু সংরক্ষিত
    2. ৬৫,০০০টি
    3. মাত্র ২টি
    4. অসীম

21. **প্রশ্ন ২১ (Chapter ৫ — Beginner):** Virtual Server Instance (VSI) বেছে নেওয়ার সময় প্রথমে কী দেখবেন?
    1. শুধু নাম
    2. CPU, memory, storage ও workload-এর চাহিদা
    3. শুধু region-এর নাম
    4. শুধু OS-এর রং

22. **প্রশ্ন ২২ (Chapter ৫ — Intermediate):** একটি VSI-তে হঠাৎ CPU ১০০% হলে প্রথমে কী করবেন?
    1. সঙ্গে সঙ্গে বড় instance-এ বদলে ফেলা
    2. কোন process, traffic নাকি ভুল কনফিগারেশন তা নির্ণয় করা
    3. সার্ভার মুছে ফেলা
    4. সব log মুছে ফেলা

23. **প্রশ্ন ২৩ (Chapter ৫ — Intermediate):** Stateless অ্যাপের জন্য instance বদলানো সহজ কেন?
    1. ডেটা instance-এর ভেতরে থাকে না
    2. ডেটা শুধু RAM-এ থাকে
    3. কোনো ডেটাবেস লাগে না
    4. instance কখনো বন্ধ হয় না

24. **প্রশ্ন ২৪ (Chapter ৫ — Advanced):** Prod ডেটাবেসের VSI-তে patch দেওয়ার আগে সবচেয়ে গুরুত্বপূর্ণ পদক্ষেপ কোনটি?
    1. সরাসরি patch চালানো
    2. backup/snapshot যাচাই ও rollback পরিকল্পনা
    3. সবাইকে logout করানো
    4. monitoring বন্ধ করা

25. **প্রশ্ন ২৫ (Chapter ৫ — Beginner):** Bare metal কখন virtual server-এর চেয়ে যুক্তিযুক্ত হতে পারে?
    1. সবসময়
    2. নির্দিষ্ট hardware, licensing বা performance isolation দরকার হলে
    3. কখনোই নয়
    4. শুধু ছোট ওয়েবসাইটে

26. **প্রশ্ন ২৬ (Chapter ৬ — Beginner):** Object storage কখন block/file storage-এর চেয়ে ভালো?
    1. ডেটাবেসের transaction-এ
    2. ছবি, backup, log-এর মতো বড় unstructured ডেটায়
    3. OS boot disk-এ
    4. খুব কম latency লেখায়

27. **প্রশ্ন ২৭ (Chapter ৬ — Intermediate):** Backup-এর 3-2-1 নিয়মের সারমর্ম কী?
    1. ৩টি কপি, ২ ধরনের মাধ্যম, ১টি off-site
    2. ৩ দিনে ২ বার backup
    3. ১টি কপি যথেষ্ট
    4. শুধু cloud-এ রাখা

28. **প্রশ্ন ২৮ (Chapter ৬ — Intermediate):** Object storage-এর bucket public হয়ে গেলে কী ঝুঁকি?
    1. কোনো ঝুঁকি নেই
    2. সংবেদনশীল ডেটা যে কেউ পড়তে পারে
    3. শুধু গতি কমে
    4. শুধু খরচ কমে

29. **প্রশ্ন ২৯ (Chapter ৬ — Advanced):** ডেটাবেস backup restore পরীক্ষা না করলে কী সমস্যা হতে পারে?
    1. কোনো সমস্যা নেই
    2. দরকারের সময় backup কাজ না-ও করতে পারে
    3. backup দ্রুত হয়
    4. খরচ কমে যায়

30. **প্রশ্ন ৩০ (Chapter ৬ — Beginner):** Storage class বেছে নেওয়ার সময় কী বিবেচ্য?
    1. ডেটা কতবার access হয় ও কতদিন রাখতে হবে
    2. শুধু bucket-এর নাম
    3. শুধু region
    4. শুধু ফাইলের রং

31. **প্রশ্ন ৩১ (Chapter ৭ — Beginner):** Managed ডেটাবেস ব্যবহারের প্রধান সুবিধা কী?
    1. backup, patch ও high availability-এর অনেক কাজ provider করে
    2. কোনো খরচ নেই
    3. ডেটা হারানো যায় না
    4. কোড লিখতে হয় না

32. **প্রশ্ন ৩২ (Chapter ৭ — Intermediate):** Read-heavy অ্যাপে ডেটাবেস ধীর হলে প্রথমে কী দেখবেন?
    1. সঙ্গে সঙ্গে ডেটাবেস বড় করা
    2. query, index ও connection ব্যবহার
    3. অ্যাপ মুছে ফেলা
    4. সব টেবিল মুছে ফেলা

33. **প্রশ্ন ৩৩ (Chapter ৭ — Intermediate):** Connection pool শেষ হয়ে গেলে সাধারণত কী হয়?
    1. অ্যাপ দ্রুত হয়
    2. নতুন request অপেক্ষা করে বা ব্যর্থ হয়
    3. ডেটা দ্বিগুণ হয়
    4. কিছুই হয় না

34. **প্রশ্ন ৩৪ (Chapter ৭ — Advanced):** Multi-region ডেটাবেসে লেখার latency বাড়ার সম্ভাব্য কারণ কোনটি?
    1. replication ও consistency প্রয়োজন
    2. শুধু অ্যাপের নাম
    3. শুধু CPU-এর রং
    4. ডেটা কম থাকা

35. **প্রশ্ন ৩৫ (Chapter ৭ — Beginner):** Production ডেটাবেসে সরাসরি manual UPDATE চালানোর আগে কী করবেন?
    1. সরাসরি চালাবেন
    2. backup, transaction ও প্রভাব যাচাই করবেন
    3. সবাইকে জানাবেন না
    4. log বন্ধ করবেন

36. **প্রশ্ন ৩৬ (Chapter ৮ — Beginner):** Kubernetes-এ Pod কী?
    1. একটি সার্ভার
    2. এক বা একাধিক container-এর সবচেয়ে ছোট deployable একক
    3. একটি ডেটাবেস
    4. একটি load balancer

37. **প্রশ্ন ৩৭ (Chapter ৮ — Intermediate):** Deployment-এ replica বাড়ালে কী হয়?
    1. একই Pod-এর একাধিক copy চলে, availability বাড়ে
    2. ডেটা মুছে যায়
    3. image বদলে যায়
    4. cluster বন্ধ হয়

38. **প্রশ্ন ৩৮ (Chapter ৮ — Intermediate):** Pod বারবার CrashLoopBackOff-এ গেলে প্রথমে কী দেখবেন?
    1. node মুছে ফেলা
    2. Pod-এর log ও event, তারপর config/env/image
    3. cluster মুছে ফেলা
    4. সব Pod মুছে ফেলা

39. **প্রশ্ন ৩৯ (Chapter ৮ — Advanced):** Stateful workload (যেমন ডেটাবেস) Kubernetes-এ চালাতে কী দরকার?
    1. শুধু Deployment
    2. StatefulSet, persistent volume ও স্থিতিশীল পরিচয়
    3. শুধু Service
    4. শুধু Ingress

40. **প্রশ্ন ৪০ (Chapter ৮ — Beginner):** Service-এর কাজ কী?
    1. Pod-কে স্থিতিশীল network ঠিকানা দেওয়া
    2. image তৈরি করা
    3. log মুছে ফেলা
    4. billing করা

বহুনির্বাচনী প্রশ্ন — ৪১–৮০

1. **প্রশ্ন ৪১ (Chapter ৯ — Beginner):** OpenShift Kubernetes-এর ওপর কী যোগ করে?
    1. শুধু দাম
    2. developer workflow, Route, Operator ও enterprise নিরাপত্তা সুবিধা
    3. শুধু storage
    4. কিছুই নয়

2. **প্রশ্ন ৪২ (Chapter ৯ — Intermediate):** OpenShift Route ও Kubernetes Ingress-এর পার্থক্য কী?
    1. একই জিনিস
    2. Route OpenShift-এর নিজস্ব edge routing; Ingress সাধারণ Kubernetes resource
    3. Route শুধু ডেটাবেসের জন্য
    4. Ingress শুধু storage-এর জন্য
3. **প্রশ্ন ৪৩ (Chapter ৯ — Intermediate):** Operator কখন ব্যবহার করবেন?
    1. জটিল অ্যাপের lifecycle (install, upgrade, backup) স্বয়ংক্রিয় করতে
    2. শুধু log দেখতে
    3. শুধু billing দেখতে
    4. কখনোই নয়
4. **প্রশ্ন ৪৪ (Chapter ৯ — Advanced):** Multi-cluster OpenShift-এ একটি অ্যাপ সব cluster-এ একইভাবে deploy করতে কী সাহায্য করে?
    1. manual kubectl প্রতিটি cluster-এ
    2. GitOps ও কেন্দ্রীয় policy/template
    3. প্রতিটি cluster-এ আলাদা কোড
    4. কোনো version control ছাড়া
5. **প্রশ্ন ৪৫ (Chapter ৯ — Beginner):** OpenShift-এ project/namespace-এর কাজ কী?
    1. resource আলাদা ও access সীমিত করা
    2. সার্ভার বন্ধ করা
    3. image মুছে ফেলা
    4. billing বাড়ানো
6. **প্রশ্ন ৪৬ (Chapter ১০ — Beginner):** Serverless-এ Code Engine বেছে নেওয়ার যুক্তি কোনটি?
    1. সার্ভার পরিচালনা না করে event বা request অনুযায়ী চলা
    2. সবসময় সার্ভার চালু রাখা
    3. manual scaling
    4. কোনো monitoring ছাড়া
7. **প্রশ্ন ৪৭ (Chapter ১০ — Intermediate):** Serverless function ঠান্ডা শুরু (cold start) হলে কী হয়?
    1. প্রথম request-এ বাড়তি latency হতে পারে
    2. function মুছে যায়
    3. খরচ শূন্য হয়
    4. ডেটা দ্বিগুণ হয়

8. **প্রশ্ন ৪৮ (Chapter ১০ — Intermediate):** দীর্ঘক্ষণ চলা, ভারী batch কাজের জন্য serverless সবসময় সেরা নয় কেন?
    1. timeout, memory ও খরচের সীমা থাকতে পারে
    2. serverless সব কাজেই সেরা
    3. batch কাজ নিষিদ্ধ
    4. কোড লেখা যায় না
9. **প্রশ্ন ৪৯ (Chapter ১০ — Advanced):** IBM Cloud Functions-কে legacy বলা হয় — নতুন serverless কাজে কী করবেন?
    1. Functions-এই নতুন কাজ শুরু করবেন
    2. Code Engine-এর মতো বর্তমান প্ল্যাটফর্ম যাচাই করবেন
    3. serverless বাদ দেবেন
    4. কোনো ডকুমেন্টেশন দেখবেন না
10. **প্রশ্ন ৫০ (Chapter ১০ — Beginner):** Event-driven architecture-এ trigger-এর কাজ কী?
    1. ঘটনা ঘটলে কাজ শুরু করা
    2. সার্ভার বন্ধ করা
    3. ডেটা মুছে ফেলা
    4. billing করা
11. **প্রশ্ন ৫১ (Chapter ১১ — Beginner):** CI/CD pipeline-এর মূল উদ্দেশ্য কী?
    1. কোড লুকানো
    2. পরীক্ষা, build ও deploy স্বয়ংক্রিয় ও পুনরাবৃত্তিযোগ্য করা
    3. manual কাজ বাড়ানো
    4. log মুছে ফেলা
12. **প্রশ্ন ৫২ (Chapter ১১ — Intermediate):** Production deploy-এর আগে staging-এ কী যাচাই করবেন?
    1. শুধু UI-এর রং
    2. functional, integration ও rollback পরীক্ষা
    3. শুধু ফাইলের নাম
    4. কিছুই নয়
13. **প্রশ্ন ৫৩ (Chapter ১১ — Intermediate):** Blue-green deployment-এর সুবিধা কী?
    1. নতুন সংস্করণ পরীক্ষা করে দ্রুত traffic বদলানো ও rollback সহজ
    2. deploy ধীর করা
    3. ডেটা মুছে ফেলা
    4. দুটি সংস্করণ একসঙ্গে চালানো যায় না

14. **প্রশ্ন ৫৪ (Chapter ১১ — Advanced):** Canary deployment কখন উপযুক্ত?
    1. সব ব্যবহারকারীকে একসঙ্গে ঝুঁকিতে ফেলতে চাইলে
    2. ছোট অংশে নতুন সংস্করণ পরীক্ষা করে ধীরে ধীরে বাড়াতে চাইলে
    3. কখনোই নয়
    4. শুধু ডেটাবেস migration-এ

15. **প্রশ্ন ৫৫ (Chapter ১১ — Beginner):** Pipeline-এ secret (API key) রাখার নিরাপদ উপায় কোনটি?
    1. Git-এ plain text-এ
    2. secret manager / pipeline secret store-এ
    3. Slack-এ পাঠানো
    4. কোডে hardcode করা

16. **প্রশ্ন ৫৬ (Chapter ১২ — Beginner):** Terraform-এর মূল ধারণা কী?
    1. manual ক্লিক করে resource বানানো
    2. Infrastructure as Code — ঘোষণামূলকভাবে resource সংজ্ঞায়িত করা
    3. শুধু অ্যাপ কোড লেখা
    4. শুধু monitoring

17. **প্রশ্ন ৫৭ (Chapter ১২ — Intermediate):** Terraform state ফাইলের কাজ কী?
    1. কোডের রং ঠিক করা
    2. বাস্তব infrastructure-এর সঙ্গে কনফিগারেশনের মিল রাখা
    3. billing করা
    4. log সংরক্ষণ

18. **প্রশ্ন ৫৮ (Chapter ১২ — Intermediate):** State ফাইল দলে শেয়ার করার নিরাপদ উপায় কোনটি?
    1. ইমেইলে পাঠানো
    2. remote backend-এ lock সহ সংরক্ষণ
    3. প্রত্যেকে আলাদা local state
    4. Git-এ secret সহ commit

19. **প্রশ্ন ৫৯ (Chapter ১২ — Advanced):** কেউ console থেকে resource বদলে ফেললে Terraform কী করবে?
    1. কিছুই করবে না
    2. plan-এ drift দেখাবে; apply করলে কনফিগারেশন অনুযায়ী ফেরত আনতে পারে
    3. স্বয়ংক্রিয়ভাবে মুছে ফেলবে
    4. state মুছে ফেলবে

20. **প্রশ্ন ৬০ (Chapter ১২ — Beginner):** Terraform module ব্যবহারের সুবিধা কী?
    1. কোড পুনঃব্যবহার ও মান বজায় রাখা

2. কোড লুকানো
3. খরচ বাড়ানো
4. state মুছে ফেলা

21. **প্রশ্ন ৬১ (Chapter ১৩ — Beginner):** Ansible-এর মূল কাজ কী?
    1. container image তৈরি
    2. সার্ভার কনফিগারেশন ও অ্যাপ deploy স্বয়ংক্রিয় করা
    3. শুধু billing
    4. শুধু DNS

22. **প্রশ্ন ৬২ (Chapter ১৩ — Intermediate):** Ansible playbook idempotent হওয়া মানে কী?
    1. প্রতিবার চালালে ভিন্ন ফল
    2. একাধিকবার চালালেও একই কাঙ্ক্ষিত অবস্থা থাকে
    3. শুধু একবার চালানো যায়
    4. কখনো কাজ করে না

23. **প্রশ্ন ৬৩ (Chapter ১৩ — Intermediate):** Inventory-তে host ভাগ করার সুবিধা কী?
    1. group অনুযায়ী ভিন্ন variable ও play চালানো যায়
    2. host লুকানো যায়
    3. খরচ বাড়ে
    4. কিছুই নয়

24. **প্রশ্ন ৬৪ (Chapter ১৩ — Advanced):** Terraform ও Ansible একসঙ্গে ব্যবহারের যুক্তিযুক্ত ভাগ কী?
    1. Terraform resource তৈরি করে; Ansible তার ভেতরের কনফিগারেশন করে
    2. দুটোই একই কাজ, একটিকে বাদ দেওয়া উচিত
    3. Ansible resource তৈরি করে; Terraform কনফিগার করে
    4. কোনো ভাগ নেই

25. **প্রশ্ন ৬৫ (Chapter ১৩ — Beginner):** Ansible vault কিসের জন্য?
    1. password/secret এনক্রিপ্ট করে রাখা
    2. image সংরক্ষণ
    3. log মুছে ফেলা
    4. billing করা

26. **প্রশ্ন ৬৬ (Chapter ১৪ — Beginner):** Monitoring, logging ও tracing-এর মধ্যে tracing কী দেয়?
    1. শুধু CPU গ্রাফ
    2. একটি request বিভিন্ন সার্ভিসে কীভাবে যায় তার পথ
    3. শুধু disk ব্যবহার

4. শুধু billing

27. **প্রশ্ন ৬৭ (Chapter ১৪ — Intermediate):** Alert-এর জন্য সবচেয়ে ভালো signal কোনটি?
    1. সব metric-এ alert
    2. ব্যবহারকারীর অভিজ্ঞতা প্রভাবিত করে এমন SLI/SLO-ভিত্তিক alert
    3. শুধু disk ৯০% হলে
    4. প্রতি মিনিটে ইমেইল
28. **প্রশ্ন ৬৮ (Chapter ১৪ — Intermediate):** Log-এ sensitive তথ্য (password, token) থাকলে কী করবেন?
    1. রেখে দেবেন
    2. redact/mask করবেন ও সংগ্রহের নীতি ঠিক করবেন
    3. সব log public করবেন
    4. log বন্ধ করবেন
29. **প্রশ্ন ৬৯ (Chapter ১৪ — Advanced):** একটি API হঠাৎ ধীর — metric ঠিক, কিন্তু ব্যবহারকারী অভিযোগ করছে। প্রথমে কী দেখবেন?
    1. শুধু CPU
    2. trace ও latency-এর percentile (p95/p99), downstream নির্ভরতা
    3. সার্ভার মুছে ফেলা
    4. billing
30. **প্রশ্ন ৭০ (Chapter ১৪ — Beginner):** Prometheus ও Grafana-এর ভূমিকা কী?
    1. Prometheus metric সংগ্রহ করে; Grafana তা দেখায় ও alert করে
    2. দুটোই ডেটাবেস
    3. দুটোই load balancer
    4. কোনোটিই monitoring নয়
31. **প্রশ্ন ৭১ (Chapter ১৫ — Beginner):** High availability (HA) বলতে কী বোঝায়?
    1. একটিমাত্র সার্ভার
    2. কোনো অংশ ব্যর্থ হলেও সার্ভিস চালু থাকা
    3. সবসময় বন্ধ থাকা
    4. শুধু backup রাখা
32. **প্রশ্ন ৭২ (Chapter ১৫ — Intermediate):** Multi-zone deploy-এর সুবিধা কী?
    1. একটি zone ব্যর্থ হলেও অন্য zone থেকে সার্ভিস চলে
    2. খরচ শূন্য হয়
    3. latency সবসময় কমে
    4. ডেটা মুছে যায়

33. **প্রশ্ন ৭৩ (Chapter ১৫ — Intermediate):** RTO ও RPO-এর পার্থক্য কী?

    1. একই জিনিস
    2. RTO: কতক্ষণে ফিরতে হবে; RPO: কতটুকু ডেটা হারানো acceptable
    3. RTO ডেটা; RPO সময়
    4. কোনোটিই disaster recovery নয়

34. **প্রশ্ন ৭৪ (Chapter ১৫ — Advanced):** Disaster recovery পরীক্ষা না করলে কী ঝুঁকি?

    1. কোনো ঝুঁকি নেই
    2. আসল বিপদে runbook কাজ না-ও করতে পারে
    3. খরচ কমে
    4. backup দ্রুত হয়

35. **প্রশ্ন ৭৫ (Chapter ১৫ — Beginner):** Load balancer-এর কাজ কী?

    1. traffic একাধিক healthy instance-এ ভাগ করা
    2. ডেটা মুছে ফেলা
    3. image তৈরি করা
    4. billing করা

36. **প্রশ্ন ৭৬ (Chapter ১৬ — Beginner):** Cloud নিরাপত্তায় encryption at rest বলতে কী?

    1. ডেটা চলাচলের সময় এনক্রিপশন
    2. সংরক্ষিত ডেটা এনক্রিপ্ট করা
    3. password মুছে ফেলা
    4. কিছুই নয়

37. **প্রশ্ন ৭৭ (Chapter ১৬ — Intermediate):** Customer-managed key (BYOK/HYOK ধারণা) ব্যবহারের সুবিধা কী?

    1. key-এর নিয়ন্ত্রণ গ্রাহকের হাতে বেশি থাকে
    2. key হারিয়ে গেলে ভালো
    3. encryption বন্ধ হয়
    4. খরচ শূন্য হয়

38. **প্রশ্ন ৭৮ (Chapter ১৬ — Intermediate):** Container image-এ vulnerability পাওয়া গেলে কী করবেন?

    1. সেই image-ই prod-এ চালাবেন
    2. patch করা base image-এ rebuild করে আবার scan করবেন
    3. scan বন্ধ করবেন
    4. log মুছে ফেলবেন

39. **প্রশ্ন ৭৯ (Chapter ১৬ — Advanced):** Zero trust নীতির সারমর্ম কী?
    1. ভেতরের network-কে সবসময় বিশ্বাস করা
    2. কাউকে স্বয়ংক্রিয়ভাবে বিশ্বাস না করে প্রতিটি access যাচাই করা
    3. কোনো authentication নয়
    4. সব port খোলা রাখা
40. **প্রশ্ন ৮০ (Chapter ১৬ — Beginner):** Security Group-এ 0.0.0.0/0 দিয়ে SSH খোলা কেন ঝুঁকিপূর্ণ?
    1. কোনো ঝুঁকি নেই
    2. ইন্টারনেটের যে কেউ SSH চেষ্টা করতে পারে
    3. গতি বাড়ে
    4. খরচ কমে

বহুনির্বাচনী প্রশ্ন — ৮১–১২০

1. **প্রশ্ন ৮১ (Chapter ১৭ — Beginner):** FinOps-এর মূল লক্ষ্য কী?
    1. শুধু খরচ কমানো
    2. খরচের স্বচ্ছতা, দায়িত্ব ও মূল্যের ভারসাম্য
    3. সব resource মুছে ফেলা
    4. billing লুকানো
2. **প্রশ্ন ৮২ (Chapter ১৭ — Intermediate):** একটি idle VM মাসের পর মাস চলছে। প্রথম পদক্ষেপ কী?
    1. সঙ্গে সঙ্গে মুছে ফেলা
    2. ব্যবহার যাচাই করে right-size, stop বা মুছে ফেলার সিদ্ধান্ত
    3. বড় করা
    4. অপেক্ষা করা
3. **প্রশ্ন ৮৩ (Chapter ১৭ — Intermediate):** Tagging খরচ নিয়ন্ত্রণে কীভাবে সাহায্য করে?
    1. টিম/প্রজেক্ট/পরিবেশ অনুযায়ী খরচ ভাগ করে দেখা যায়
    2. গতি বাড়ে
    3. ডেটা মুছে যায়
    4. কিছুই নয়
4. **প্রশ্ন ৮৪ (Chapter ১৭ — Advanced):** হঠাৎ billing বেড়ে গেলে প্রথমে কী দেখবেন?
    1. সব resource মুছে ফেলা
    2. কোন সার্ভিস/resource-এ বৃদ্ধি, কখন থেকে, কী trigger করেছে
    3. password বদলানো
    4. monitoring বন্ধ করা

5. **প্রশ্ন ৮৫ (Chapter ১৭ — Beginner):** Reserved/committed ব্যবহার কখন যুক্তিযুক্ত?
    1. অস্থির, স্বল্পমেয়াদি workload-এ
    2. স্থিতিশীল, দীর্ঘমেয়াদি baseline workload-এ
    3. কখনোই নয়
    4. শুধু পরীক্ষায়
6. **প্রশ্ন ৮৬ (Chapter ১৮ — Beginner):** Multi-cloud বলতে কী বোঝায়?
    1. একটিমাত্র cloud ব্যবহার
    2. একাধিক cloud provider-এর সার্ভিস একসঙ্গে ব্যবহার
    3. শুধু on-premises
    4. কোনো cloud নয়
7. **প্রশ্ন ৮৭ (Chapter ১৮ — Intermediate):** Multi-cloud-এর প্রধান ঝুঁকি কোনটি?
    1. জটিলতা, ভিন্ন API/নিরাপত্তা মডেল ও ডেটা স্থানান্তর খরচ
    2. খরচ সবসময় কমে
    3. কোনো ঝুঁকি নেই
    4. গতি সবসময় বাড়ে
8. **প্রশ্ন ৮৮ (Chapter ১৮ — Intermediate):** Vendor lock-in কমাতে কী সাহায্য করে?
    1. শুধু একটি provider-এর proprietary সার্ভিসে সব বাঁধা
    2. খোলা মান (Kubernetes, Terraform, open API) ও portable design
    3. কোনো documentation ছাড়া
    4. সব ডেটা এক জায়গায় আটকে রাখা
9. **প্রশ্ন ৮৯ (Chapter ১৮ — Advanced):** একটি workload-এর ডেটা residency আইনত একটি দেশে সীমিত। Multi-cloud সিদ্ধান্তে কী করবেন?
    1. সবচেয়ে সস্তা region বেছে নেবেন
    2. ডেটা কোথায় থাকে, কোন provider সেই প্রয়োজন মেটায় তা যাচাই করবেন
    3. আইন উপেক্ষা করবেন
    4. ডেটা public করবেন
10. **প্রশ্ন ৯০ (Chapter ১৮ — Beginner):** Hybrid cloud কখন উপযুক্ত?
    1. কিছু workload on-premises, কিছু cloud-এ রাখতে চাইলে
    2. কখনোই নয়
    3. শুধু ছোট অ্যাপে
    4. সবসময়

11. **প্রশ্ন ৯১ (Chapter ১৯ — Beginner):** Cross-cloud connectivity-তে সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা বিবেচনা কী?
    1. সব port খোলা
    2. encrypted, authenticated ও least-privilege সংযোগ
    3. password শেয়ার করা
    4. public IP ব্যবহার

12. **প্রশ্ন ৯২ (Chapter ১৯ — Intermediate):** দুই cloud-এর মধ্যে ডেটা স্থানান্তর ধীর হলে কী দেখবেন?
    1. শুধু অ্যাপের নাম
    2. bandwidth, latency, egress পথ ও ডেটার আকার/ফরম্যাট
    3. billing বন্ধ করা
    4. সার্ভার মুছে ফেলা

13. **প্রশ্ন ৯৩ (Chapter ১৯ — Intermediate):** Multi-cloud-এ identity পরিচালনার ভালো পদ্ধতি কোনটি?
    1. প্রতিটি cloud-এ আলাদা আলাদা password
    2. federated identity ও কেন্দ্রীয় নীতি
    3. একই API key সব জায়গায়
    4. কোনো identity নয়

14. **প্রশ্ন ৯৪ (Chapter ১৯ — Advanced):** একটি cloud down হলে অন্য cloud-এ failover করতে কী আগে থেকে থাকতে হবে?
    1. কিছুই নয়, তখন বানানো যাবে
    2. পরীক্ষিত runbook, replicated ডেটা ও DNS/traffic পরিকল্পনা
    3. শুধু একটি backup ফাইল
    4. শুধু একটি ফোন নম্বর

15. **প্রশ্ন ৯৫ (Chapter ১৯ — Beginner):** API gateway multi-cloud-এ কী সাহায্য করে?
    1. একটি সামঞ্জস্যপূর্ণ entry point, auth ও rate limiting দেওয়া
    2. ডেটা মুছে ফেলা
    3. billing বাড়ানো
    4. কিছুই নয়

16. **প্রশ্ন ৯৬ (Chapter ২০ — Beginner):** Compliance (যেমন GDPR ধারণা) cloud design-এ কেন গুরুত্বপূর্ণ?
    1. শুধু কাগজপত্রের জন্য
    2. ডেটা কোথায় থাকে, কে access করে ও কতদিন রাখা হয় তা নির্ধারণ করে
    3. গতি বাড়ায়
    4. খরচ শূন্য করে

17. **প্রশ্ন ৯৭ (Chapter ২০ — Intermediate):** Audit log-এর প্রধান উদ্দেশ্য কী?
    1. কে, কখন, কী বদলেছে তার প্রমাণ রাখা
    2. log মুছে ফেলা
    3. গতি বাড়ানো
    4. billing করা
18. **প্রশ্ন ৯৮ (Chapter ২০ — Intermediate):** Production-এ ব্যক্তিগত ডেটা (PII) log-এ এলে কী করবেন?
    1. রেখে দেবেন
    2. mask/redact করবেন ও retention নীতি ঠিক করবেন
    3. public করবেন
    4. log বন্ধ করবেন
19. **প্রশ্ন ৯৯ (Chapter ২০ — Advanced):** একটি নতুন regulation ডেটা একটি নির্দিষ্ট region-এর বাইরে যেতে নিষেধ করলে কী করবেন?
    1. উপেক্ষা করবেন
    2. data residency, replication ও backup-এর অবস্থান পুনরায় নকশা করবেন
    3. ডেটা মুছে ফেলবেন
    4. সব region বন্ধ করবেন
20. **প্রশ্ন ১০০ (Chapter ২০ — Beginner):** Policy as Code বলতে কী বোঝায়?
    1. নীতি মুখে বলা
    2. নিরাপত্তা/compliance নিয়ম কোড হিসেবে লিখে স্বয়ংক্রিয়ভাবে যাচাই করা
    3. কোড মুছে ফেলা
    4. কিছুই নয়
21. **প্রশ্ন ১০১ (Chapter ২১ — Beginner):** GitOps-এর মূল ধারণা কী?
    1. Git-এ শুধু অ্যাপ কোড রাখা
    2. Git-কে infrastructure ও deploy-এর সত্যের উৎস (source of truth) করা
    3. manual deploy
    4. Git মুছে ফেলা
22. **প্রশ্ন ১০২ (Chapter ২১ — Intermediate):** GitOps-এ production-এ পরিবর্তন কীভাবে আসা উচিত?
    1. সরাসরি console থেকে
    2. Git-এ commit/PR ও review-এর মাধ্যমে, agent তা প্রয়োগ করে
    3. ফোনে জানিয়ে
    4. ইমেইলে

23. **প্রশ্ন ১০৩ (Chapter ২১ — Intermediate):** Argo CD/Flux ধরনের টুলের সুবিধা কী?
    1. cluster-এর অবস্থা Git-এর সঙ্গে মিলিয়ে রাখা ও drift দেখানো
    2. Git মুছে ফেলা
    3. কোড লেখা বন্ধ করা
    4. billing করা

24. **প্রশ্ন ১০৪ (Chapter ২১ — Advanced):** GitOps-এ secret কীভাবে পরিচালনা করবেন?
    1. Git-এ plain text-এ
    2. encrypted secret বা external secret manager দিয়ে
    3. Slack-এ
    4. কোডে hardcode করে

25. **প্রশ্ন ১০৫ (Chapter ২১ — Beginner):** Infrastructure-এ drift বলতে কী?
    1. Git ও বাস্তব অবস্থার মধ্যে পার্থক্য
    2. নেটওয়ার্কের গতি
    3. ডেটার আকার
    4. কিছুই নয়

26. **প্রশ্ন ১০৬ (Chapter ২২ — Beginner):** Chaos engineering-এর উদ্দেশ্য কী?
    1. production ভেঙে ফেলা
    2. নিয়ন্ত্রিতভাবে ব্যর্থতা ঘটিয়ে resilience যাচাই করা
    3. log মুছে ফেলা
    4. খরচ বাড়ানো

27. **প্রশ্ন ১০৭ (Chapter ২২ — Intermediate):** Chaos পরীক্ষা চালানোর আগে কী থাকতে হবে?
    1. কিছুই নয়
    2. hypothesis, blast radius সীমা, rollback ও monitoring
    3. সব alert বন্ধ
    4. backup মুছে ফেলা

28. **প্রশ্ন ১০৮ (Chapter ২২ — Intermediate):** Load testing-এর সময় কোন metric সবচেয়ে গুরুত্বপূর্ণ?
    1. শুধু CPU
    2. latency, error rate, throughput ও resource ব্যবহার একসঙ্গে
    3. শুধু ফাইলের নাম
    4. শুধু billing

29. **প্রশ্ন ১০৯ (Chapter ২২ — Advanced):** একটি সার্ভিস p99 latency-তে খারাপ, কিন্তু গড় latency ভালো। এর অর্থ কী?
    1. সব request ভালো
    2. অল্প কিছু request খুব ধীর — tail latency সমস্যা, তদন্ত দরকার
    3. কোনো সমস্যা নেই
    4. গড়ই যথেষ্ট

30. **প্রশ্ন ১১০ (Chapter ২২ — Beginner):** Performance test production-এর মতো পরিবেশে কেন করবেন?
    1. ফল যেন বাস্তবের কাছাকাছি হয়
    2. সময় নষ্ট করতে
    3. খরচ বাড়াতে
    4. কোনো কারণ নেই

31. **প্রশ্ন ১১১ (Chapter ২৩ — Beginner):** Incident response-এ প্রথম করণীয় কী?
    1. দোষ খোঁজা
    2. প্রভাব সীমিত করা, যোগাযোগ শুরু করা ও তথ্য সংগ্রহ করা
    3. সব log মুছে ফেলা
    4. অপেক্ষা করা

32. **প্রশ্ন ১১২ (Chapter ২৩ — Intermediate):** Blameless postmortem-এর উদ্দেশ্য কী?
    1. কাউকে শাস্তি দেওয়া
    2. ব্যক্তিকে নয়, ব্যবস্থার দুর্বলতা খুঁজে প্রতিরোধমূলক ব্যবস্থা নেওয়া
    3. ঘটনা লুকানো
    4. কিছুই না করা

33. **প্রশ্ন ১১৩ (Chapter ২৩ — Intermediate):** Runbook-এর মূল্য কখন বোঝা যায়?
    1. শান্ত সময়ে
    2. চাপের মধ্যে incident-এ দ্রুত, ধারাবাহিক পদক্ষেপ নিতে
    3. কখনোই নয়
    4. শুধু audit-এ

34. **প্রশ্ন ১১৪ (Chapter ২৩ — Advanced):** একই incident বারবার ঘটলে কী করবেন?
    1. প্রতিবার একইভাবে ঠিক করবেন
    2. মূল কারণ (root cause) খুঁজে স্থায়ী প্রতিকার ও প্রতিরোধমূলক কাজ করবেন
    3. monitoring বন্ধ করবেন
    4. ঘটনা লুকাবেন

35. **প্রশ্ন ১১৫ (Chapter ২৩ — Beginner):** On-call rotation-এর উদ্দেশ্য কী?

    1. একজনকে সবসময় জাগিয়ে রাখা
    2. দায়িত্ব ভাগ করে দ্রুত সাড়া দেওয়া ও ক্লান্তি কমানো
    3. কাজ বাড়ানো
    4. কিছুই নয়

36. **প্রশ্ন ১১৬ (Chapter ২৪ — Beginner):** Capstone প্রজেক্টে প্রথমে কী ঠিক করবেন?

    1. শুধু টুলের নাম
    2. সমস্যা, ব্যবহারকারী, সাফল্যের মাপকাঠি ও সীমা
    3. শুধু রং
    4. কিছুই নয়

37. **প্রশ্ন ১১৭ (Chapter ২৪ — Intermediate):** Capstone-এ multi-cloud বেছে নেওয়ার যুক্তি কীভাবে দেখাবেন?

    1. “সবাই করছে” বলে
    2. প্রয়োজন, খরচ, ঝুঁকি ও trade-off-এর প্রমাণ দিয়ে
    3. দাম না দেখে
    4. কোনো যুক্তি ছাড়া

38. **প্রশ্ন ১১৮ (Chapter ২৪ — Intermediate):** Capstone ডেমোতে ব্যর্থতা দেখালে কী করবেন?

    1. লুকাবেন
    2. কী শিখেছেন, কী বদলেছেন ও কীভাবে যাচাই করেছেন তা দেখাবেন
    3. প্রজেক্ট বাদ দেবেন
    4. কিছুই বলবেন না

39. **প্রশ্ন ১১৯ (Chapter ২৪ — Advanced):** Production-প্রস্তুত capstone-এর সবচেয়ে গুরুত্বপূর্ণ প্রমাণ কোনটি?

    1. সুন্দর স্লাইড
    2. পরীক্ষিত deploy, rollback, monitoring, নিরাপত্তা ও খরচের প্রমাণ
    3. অনেক টুলের তালিকা
    4. দীর্ঘ ডকুমেন্ট

40. **প্রশ্ন ১২০ (Chapter ২৪ — Beginner):** শেখা চালিয়ে যাওয়ার সেরা অভ্যাস কোনটি?

    1. একবার পড়ে ভুলে যাওয়া
    2. ছোট ছোট পরীক্ষা, নোট ও নিয়মিত অনুশীলন
    3. শুধু ভিডিও দেখা
    4. কখনো অনুশীলন না করা

Volume 14 — Assessment & Answer Guide

সংক্ষিপ্ত উত্তর প্রশ্ন — ৩০টি

প্রতিটি উত্তর ৩–৫ বাক্যে লিখুন। উত্তর ও মূল পয়েন্ট দেখুন **answer-guide**-এ।

1. **প্রশ্ন ১ (Chapter ১ — Beginner):** Shared responsibility model নিজের ভাষায় ব্যাখ্যা করুন।
2. **প্রশ্ন ২ (Chapter ২ — Beginner):** Resource group কেন dev/staging/prod আলাদা করতে সাহায্য করে?
3. **প্রশ্ন ৩ (Chapter ৩ — Intermediate):** Access Group ও সরাসরি ব্যবহারকারীকে policy দেওয়ার মধ্যে কোনটি ভালো এবং কেন?
4. **প্রশ্ন ৪ (Chapter ৩ — Intermediate):** Service ID ও ব্যক্তিগত API key-এর ব্যবহারে পার্থক্য কী?
5. **প্রশ্ন ৫ (Chapter ৪ — Beginner):** CIDR 10.10.0.0/16 ও 10.10.1.0/24-এর সম্পর্ক ব্যাখ্যা করুন।
6. **প্রশ্ন ৬ (Chapter ৪ — Intermediate):** Security Group ও NACL কখন একসঙ্গে ব্যবহার করবেন?
7. **প্রশ্ন ৭ (Chapter ৫ — Intermediate):** Stateless অ্যাপ কেন scale করা সহজ? উদাহরণ দিন।
8. **প্রশ্ন ৮ (Chapter ৬ — Intermediate):** Object storage-এ versioning ও lifecycle rule কী কাজে লাগে?
9. **প্রশ্ন ৯ (Chapter ৭ — Intermediate):** ডেটাবেস connection pool কেন সীমিত রাখা হয়? বেশি হলে কী হয়?
10. **প্রশ্ন ১০ (Chapter ৮ — Beginner):** Pod, Deployment ও Service-এর সম্পর্ক একটি উদাহরণ দিয়ে বোঝান।
11. **প্রশ্ন ১১ (Chapter ৮ — Intermediate):** Liveness ও readiness probe-এর পার্থক্য কী? ভুল ব্যবহারে কী হয়?
12. **প্রশ্ন ১২ (Chapter ৯ — Intermediate):** OpenShift Route কখন Ingress-এর চেয়ে সুবিধাজনক?
13. **প্রশ্ন ১৩ (Chapter ১০ — Intermediate):** Cold start কী এবং তা কমানোর দুটি উপায় লিখুন।
14. **প্রশ্ন ১৪ (Chapter ১১ — Intermediate):** Blue-green ও canary deployment-এর মধ্যে কখন কোনটি বেছে নেবেন?
15. **প্রশ্ন ১৫ (Chapter ১২ — Intermediate):** Terraform-এ drift কীভাবে শনাক্ত করবেন ও কী করবেন?
16. **প্রশ্ন ১৬ (Chapter ১২ — Advanced):** Remote state ও state locking কেন দলে জরুরি?
17. **প্রশ্ন ১৭ (Chapter ১৩ — Beginner):** Ansible inventory ও playbook-এর ভূমিকা সংক্ষেপে লিখুন।
18. **প্রশ্ন ১৮ (Chapter ১৪ — Intermediate):** SLI, SLO ও alert-এর সম্পর্ক ব্যাখ্যা করুন।
19. **প্রশ্ন ১৯ (Chapter ১৪ — Advanced):** গড় latency ভালো, কিন্তু p99 খারাপ — এর অর্থ ও তদন্তের পথ লিখুন।
20. **প্রশ্ন ২০ (Chapter ১৫ — Intermediate):** RTO ও RPO-এর উদাহরণ দিয়ে পার্থক্য বোঝান।
21. **প্রশ্ন ২১ (Chapter ১৬ — Intermediate):** Least privilege কীভাবে container ও IAM দুই ক্ষেত্রেই প্রয়োগ করবেন?
22. **প্রশ্ন ২২ (Chapter ১৭ — Intermediate):** Tagging ছাড়া খরচের দায় নির্ধারণ করা কঠিন কেন?
23. **প্রশ্ন ২৩ (Chapter ১৮ — Intermediate):** Vendor lock-in কমানোর তিনটি বাস্তব উপায় লিখুন।

24. **প্রশ্ন ২৪ (Chapter ১৯ — Advanced):** Cross-cloud failover পরিকল্পনায় ডেটা replication কেন সবচেয়ে কঠিন অংশ?
25. **প্রশ্ন ২৫ (Chapter ২০ — Intermediate):** Audit log ও compliance-এর সম্পর্ক সংক্ষেপে লিখুন।
26. **প্রশ্ন ২৬ (Chapter ২১ — Intermediate):** GitOps-এ Git-ই সত্যের উৎস হলে console থেকে বদলানো কেন সমস্যা?
27. **প্রশ্ন ২৭ (Chapter ২২ — Intermediate):** Chaos engineering-এ blast radius সীমিত রাখা কেন জরুরি?
28. **প্রশ্ন ২৮ (Chapter ২৩ — Intermediate):** Blameless postmortem কীভাবে ভবিষ্যৎ incident কমায়?
29. **প্রশ্ন ২৯ (Chapter ২৪ — Advanced):** Capstone প্রজেক্টে “production-প্রস্তুত” প্রমাণের জন্য কোন পাঁচটি জিনিস দেখাবেন?
30. **প্রশ্ন ৩০ (Chapter ২৪ — Beginner):** এই ভলিউম থেকে আপনার সবচেয়ে বড় তিনটি শেখা কী? প্রতিটি এক বাক্যে লিখুন।

Volume 14 — Assessment & Answer Guide

Troubleshooting Scenario — ২০টি

প্রতিটি পরিস্থিতিতে বলুন: **প্রথমে কী চেক করবেন এবং কেন?** তারপর পরের দুটি পদক্ষেপ। উত্তর দেখুন **answer-guide-2**-এ।

1. **পরিস্থিতি ১ — ওয়েবসাইট খুলছে না:** TechNova-র সাইট হঠাৎ “connection timed out” দেখাচ্ছে। অ্যাপের log-এ কোনো error নেই। প্রথমে কী চেক করবেন এবং কেন?
2. **পরিস্থিতি ২ — 403 Forbidden:** একজন developer IBM Cloud console-এ resource দেখতে পাচ্ছে না, কিন্তু login ঠিক আছে। প্রথমে কী চেক করবেন এবং কেন?
3. **পরিস্থিতি ৩ — Pod CrashLoopBackOff:** নতুন deploy-এর পর একটি Pod বারবার restart হচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
4. **পরিস্থিতি ৪ — ডেটাবেস ধীর:** সকালে অ্যাপ ধীর, ডেটাবেস CPU ৯৫%। প্রথমে কী চেক করবেন এবং কেন?
5. **পরিস্থিতি ৫ — খরচ হঠাৎ বেড়েছে:** গত সপ্তাহের তুলনায় billing দ্বিগুণ। কোনো নতুন deploy হয়নি বলে দল মনে করছে। প্রথমে কী চেক করবেন এবং কেন?
6. **পরিস্থিতি ৬ — API key কাজ করছে না:** CI pipeline হঠাৎ 401 দিচ্ছে; গতকাল ঠিক ছিল। প্রথমে কী চেক করবেন এবং কেন?
7. **পরিস্থিতি ৭ — DNS সমস্যা:** কিছু ব্যবহারকারী সাইট খুলতে পারছে, কিছু পারছে না। প্রথমে কী চেক করবেন এবং কেন?
8. **পরিস্থিতি ৮ — Disk পূর্ণ:** VSI-তে “no space left on device” — অ্যাপ লিখতে পারছে না। প্রথমে কী চেক করবেন এবং কেন?
9. **পরিস্থিতি ৯ — Load balancer-এ 502:** Load balancer 502 দিচ্ছে, কিন্তু instance চলছে বলে মনে হচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
10. **পরিস্থিতি ১০ — Terraform apply ব্যর্থ:** `terraform apply` state lock error দিচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
11. **পরিস্থিতি ১১ — Object storage-এ ফাইল পাওয়া যাচ্ছে না:** আপলোড করা ছবি 404 দিচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
12. **পরিস্থিতি ১২ — VPN/Direct Link বিচ্ছিন্ন:** On-premises থেকে VPC-তে সংযোগ হঠাৎ বন্ধ। প্রথমে কী চেক করবেন এবং কেন?
13. **পরিস্থিতি ১৩ — মেমরি লিক সন্দেহ:** Pod-এর memory ধীরে ধীরে বাড়ছে, কয়েক দিন পর OOMKilled হচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
14. **পরিস্থিতি ১৪ — Deploy-এর পর ভুল সংস্করণ:** নতুন কোড deploy হয়েছে বলে pipeline বলছে, কিন্তু সাইটে পুরোনো সংস্করণ দেখাচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
15. **পরিস্থিতি ১৫ — Certificate expired:** ব্রাউজার “certificate expired” দেখাচ্ছে। প্রথমে কী চেক করবেন এবং কেন?

16. **পরিস্থিতি ১৬ — Multi-cloud latency:** IBM Cloud-এর অ্যাপ থেকে অন্য cloud-এর API ধীর। প্রথমে কী চেক করবেন এবং কেন?
17. **পরিস্থিতি ১৭ — Backup restore ব্যর্থ:** Disaster recovery মহড়ায় restore আটকে যাচ্ছে। প্রথমে কী চেক করবেন এবং কেন?
18. **পরিস্থিতি ১৮ — Alert ঝড়:** একসঙ্গে শত শত alert আসছে, কোনটি আসল বোঝা যাচ্ছে না। প্রথমে কী চেক করবেন এবং কেন?
19. **পরিস্থিতি ১৯ — Image pull ব্যর্থ:** Pod ImagePullBackOff-এ আছে। প্রথমে কী চেক করবেন এবং কেন?
20. **পরিস্থিতি ২০ — Region বিভ্রাট:** একটি region-এর সার্ভিস ধীর/অনুপলব্ধ, অন্য region ঠিক আছে। প্রথমে কী চেক করবেন এবং কেন?

Volume 14 — Assessment & Answer Guide

Production Incident Case Study — ১০টি

প্রতিটি কেস পড়ে বিশ্লেষণ প্রশ্নগুলোর উত্তর লিখুন। সব নাম, সংখ্যা ও সময় কাল্পনিক।

1. কেস ১ — ভুল Security Group-এ সাইট বন্ধ

ঘটনা: TechNova-র checkout পেজ হঠাৎ খুলছে না। **Timeline:** ১০:০২ deploy শুরু; ১০:০৫ Security Group নিয়ম বদল; ১০:০৭ ব্যবহারকারী অভিযোগ; ১০:৪০ নিয়ম ফেরত। **Impact:** ৩৩ মিনিট checkout বন্ধ, অর্ডার হারানো।

বিশ্লেষণ প্রশ্ন: (ক) মূল কারণ কী হতে পারে? (খ) কেন deploy-এর সঙ্গে network নিয়ম বদলানো ঝুঁকিপূর্ণ? (গ) ভবিষ্যতে কী প্রতিরোধমূলক ব্যবস্থা নেবেন?

2. কেস ২ — API key Git-এ ফাঁস

ঘটনা: একজন developer ভুলে API key Git repository-তে commit করে। **Timeline:** ০৯:০০ commit; ১১:৩০ অস্বাভাবিক API ব্যবহার ধরা পড়ে; ১২:০০ key revoke। **Impact:** অননুমোদিত ব্যবহার, খরচ বৃদ্ধি, key rotation।

বিশ্লেষণ প্রশ্ন: (ক) প্রথম ৩০ মিনিটে কী করা উচিত ছিল? (খ) কীভাবে ভবিষ্যতে ফাঁস রোধ করবেন? (গ) Secret rotation পরিকল্পনা কেমন হবে?

3. কেস ৩ — ডেটাবেস migration-এ ডেটা হারানো

ঘটনা: Production migration-এ একটি কলাম মুছে যায়। **Timeline:** ২২:০০ backup যাচাই ছাড়া migration; ২২:১৫ error; ২৩:৩০ পুরোনো backup থেকে আংশিক restore। **Impact:** কয়েক ঘণ্টার ডেটা হারানো।

বিশ্লেষণ প্রশ্ন: (ক) কোন ধাপগুলো বাদ পড়েছিল? (খ) নিরাপদ migration-এর checklist কী হবে? (গ) RPO এখানে কীভাবে প্রযোজ্য?

4. কেস ৪ — একটি zone down, অ্যাপ down

ঘটনা: একটি availability zone-এ সমস্যা; TechNova-র অ্যাপ পুরো বন্ধ। **Timeline:** ১৪:০০ zone সমস্যা; ১৪:০৫ অ্যাপ বন্ধ; ১৫:২০ manual failover। **Impact:** ৮০ মিনিট downtime।

বিশ্লেষণ প্রশ্ন: (ক) কেন single-zone design ব্যর্থ হলো? (খ) Multi-zone design কেমন হবে? (গ) Failover কীভাবে স্বয়ংক্রিয় করবেন?

5. কেস ৫ — খরচের বিস্ফোরণ

ঘটনা: একটি ভুল loop-এর কারণে object storage-এ লাখ লাখ request ও egress বাড়ে। **Timeline:** সোমবার deploy; বুধবার billing alert; বৃহস্পতিবার loop বন্ধ। **Impact:** অস্বাভাবিক বিল, budget শেষ।

বিশ্লেষণ প্রশ্ন: (ক) কেন alert দেরিতে এলো? (খ) কোন guardrail থাকলে আগেই ধরা পড়ত? (গ) Cost alert ও budget নীতি কেমন হবে?

6. কেস ৬ — Pod-এর memory লিক

ঘটনা: একটি সার্ভিস প্রতি ২ দিনে OOMKilled হচ্ছে। **Timeline:** সপ্তাহজুড়ে restart বাড়ছে; শুক্রবার peak-এ একসঙ্গে অনেক Pod মারা যায়। **Impact:** মাঝে মাঝে ধীরতা ও error।

বিশ্লেষণ প্রশ্ন: (ক) Restart-কে কেন সমাধান ধরা যাবে না? (খ) তদন্তের ধাপ কী হবে? (গ) Resource limit ও monitoring কীভাবে ঠিক করবেন?

7. কেস ৭ — ভুল region-এ ডেটা

ঘটনা: Compliance review-এ দেখা গেল backup একটি ভিন্ন region-এ যাচ্ছে, যেখানে ডেটা রাখা নিষেধ। **Timeline:** ৩ মাস আগে backup নীতি বদল; review-এ ধরা পড়ে; ১ সপ্তাহে নীতি ঠিক করা হয়। **Impact:** Compliance ঝুঁকি, নীতি পুনর্লেখন।

বিশ্লেষণ প্রশ্ন: (ক) কীভাবে এমন ভুল হয়? (খ) Policy as Code কীভাবে সাহায্য করত? (গ) Data residency যাচাইয়ের checklist কী হবে?

8. কেস ৮ — Deploy-এর পর ভুল সংস্করণ

ঘটনা: Canary-তে ভুল image tag deploy হয়; ১০% ব্যবহারকারী পুরোনো/ভুল ফিচার পায়। **Timeline:** ১৬:০০ deploy; ১৬:২০ অভিযোগ; ১৬:৪৫ rollback। **Impact:** সীমিত ব্যবহারকারী প্রভাবিত, আস্থা কমে।

বিশ্লেষণ প্রশ্ন: (ক) Image tag-এ কী ভুল ছিল? (খ) Immutable tag ও verification কীভাবে সাহায্য করত? (গ) Canary monitoring-এ কী দেখা উচিত ছিল?

9. কেস ৯ — DNS ও certificate একসঙ্গে

ঘটনা: ডোমেইন পরিবর্তনের পর certificate মেলেনি; সাইট “not secure” দেখাচ্ছে। **Timeline:** ০৯:০০ DNS বদল; ০৯:১০ certificate error; ১১:০০ certificate পুনরায় issue। **Impact:** ২ ঘণ্টা আস্থা ও traffic ক্ষতি।

বিশ্লেষণ প্রশ্ন: (ক) DNS ও certificate-এর নির্ভরতা কী? (খ) পরিবর্তনের আগে কী যাচাই করবেন? (গ) Rollback পরিকল্পনা কেমন হবে?

10. কেস ১০ — Multi-cloud failover ব্যর্থ

ঘটনা: Primary cloud-এ বিভ্রাট; secondary cloud-এ failover চেষ্টা ব্যর্থ — ডেটা পুরোনো, DNS আটকে আছে। **Timeline:** ১৩:০০ বিভ্রাট; ১৩:৩০ failover শুরু; ১৫:০০ আংশিক সার্ভিস; ১৭:০০ primary ফেরে। **Impact:** ৪ ঘণ্টা আংশিক বিভ্রাট।

বিশ্লেষণ প্রশ্ন: (ক) Failover কেন ব্যর্থ হলো — ডেটা, DNS নাকি runbook? (খ) পরীক্ষা ছাড়া failover পরিকল্পনা কেন যথেষ্ট নয়? (গ) নিয়মিত মহড়ার পরিকল্পনা কেমন হবে?

Volume 14 — Assessment & Answer Guide

চূড়ান্ত পরীক্ষা — ১০০ নম্বর, ৩ ঘণ্টা

নির্দেশনা: নোট, বই ও ইন্টারনেট বন্ধ রেখে পরীক্ষা দিন। সব বিভাগের উত্তর দিতে হবে। সময় ভাগ করে নিন। কাল্পনিক প্রেক্ষাপট TechNova; কোনো বাস্তব দাম, limit বা credential ব্যবহার করবেন না।

বিভাগ	প্রশ্ন	নম্বর	সময় (আনুমানিক)
ক — MCQ	২০টি	২০	৩০ মিনিট
খ — সংক্ষিপ্ত উত্তর	৫টি	১৫	৩০ মিনিট
গ — Scenario	৩টি	১৫	৩০ মিনিট
ঘ — Hands-on design	২টি	৩০	৬০ মিনিট
ঙ — Incident বিশ্লেষণ	১টি	২০	৩০ মিনিট
মোট	**—**	**১০০**	**৩ ঘণ্টা**

বিভাগ ক — MCQ (২০ × ১ = ২০)

MCQ অংশ থেকে পরীক্ষক ২০টি প্রশ্ন বেছে নেবেন (প্রতিটি অধ্যায় থেকে অন্তত একটি নয় — সব অধ্যায় মিলিয়ে)। নিজে অনুশীলনের জন্য যেকোনো ২০টি বেছে নিয়ে উত্তর দিন।

বিভাগ খ — সংক্ষিপ্ত উত্তর (৫ × ৩ = ১৫)

1. Least privilege IAM-এ কীভাবে প্রয়োগ করবেন? একটি উদাহরণ দিন।
2. Security Group ও NACL-এর পার্থক্য ও একসঙ্গে ব্যবহারের যুক্তি লিখুন।
3. Terraform-এ drift কী এবং তা কীভাবে ঠিক করবেন?
4. RTO ও RPO-এর উদাহরণসহ পার্থক্য লিখুন।
5. GitOps-এ Git সত্যের উৎস হলে console থেকে বদলানো কেন সমস্যা?

বিভাগ গ — Scenario (৩ × ৫ = ১৫)

1. Production সাইট “connection timed out” — প্রথমে কী চেক করবেন এবং কেন? পরের পদক্ষেপগুলো লিখুন।
2. Pod CrashLoopBackOff — log, event, config ও image যাচাইয়ের ক্রম লিখুন।
3. Billing হঠাৎ দ্বিগুণ — কোন সার্ভিস, কখন থেকে, কী trigger করেছে তা কীভাবে খুঁজবেন?

বিভাগ ঘ — Hands-on Design (২ × ১৫ = ৩০)

কাজ ১ (১৫ নম্বর): TechNova-র একটি stateless web অ্যাপ ও PostgreSQL ডেটাবেস IBM Cloud-এ নকশা করুন — VPC (২টি zone, public/private subnet), Security Group নিয়ম, managed ডেটাবেস, load balancer, backup ও monitoring। একটি নেটওয়ার্ক ডায়াগ্রাম (ASCII/তালিকা) ও Terraform module কাঠামো দিন।

কাজ ২ (১৫ নম্বর): একই অ্যাপের multi-cloud disaster recovery পরিকল্পনা লিখুন — RTO ৪ ঘণ্টা, RPO ১ ঘণ্টা ধরে: ডেটা replication, DNS/traffic, failover ধাপ, পরীক্ষার সূচি ও rollback। কোন trade-off মেনে নিচ্ছেন তা স্পষ্ট করুন।

বিভাগ ঙ — Incident বিশ্লেষণ (২০ নম্বর)

Case Study ৪ (একটি zone down, অ্যাপ down) পড়ে লিখুন: (ক) timeline ও মূল কারণ (৫), (খ) তাৎক্ষণিক প্রতিকার (৫), (গ) স্থায়ী প্রতিরোধমূলক ব্যবস্থা — architecture, runbook, পরীক্ষা (১০)।

পাসের ধারণা: ৭০ বা তার বেশি — production-প্রস্তুত ভিত্তি; ৫০–৬৯ — দুর্বল বিভাগে ফিরে যান; ৫০-এর নিচে — সংশ্লিষ্ট অধ্যায়গুলো আবার পড়ুন ও quick reference অনুশীলন করুন।

Volume 14 — Assessment & Answer Guide

উত্তর নির্দেশিকা — MCQ ১–৬০ ও সংক্ষিপ্ত উত্তর

প্রতিটি উত্তরের সঙ্গে সংক্ষিপ্ত ব্যাখ্যা দেওয়া হয়েছে। শুধু অক্ষর মুখস্থ করবেন না — ব্যাখ্যাটি বুঝুন।

MCQ উত্তর ১–৬০

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Workload বোঝা ছাড়া সাইজিং অনুমান মাত্র।
২	খ	ডেটা, compliance, latency ও খরচ একসঙ্গে বিবেচ্য।
৩	গ	স্তর অনুযায়ী দায়িত্ব ভাগ হয়।
৪	খ	ওঠানামা করা traffic-এ auto-scaling ও stateless design উপযুক্ত।
৫	খ	IaaS/PaaS/SaaS পার্থক্য পরিচালনার দায়িত্বে।
৬	খ	Resource group সংগঠন, access ও billing আলাদা করে।
৭	খ	পরিবেশভিত্তিক আলাদা group ও policy নিরাপদ।
৮	ক	`ibmcloud target` বর্তমান target দেখায়।
৯	খ	Group ও tag ছাড়া খরচের দায় জানা যায় না।
১০	ঘ	ড্যাশবোর্ডের থিম region নির্বাচনের বিষয় নয়।
১১	খ	Least privilege: যতটুকু দরকার ততটুকুই।
১২	খ	সীমিত scope-এ Viewer role-ই যথেষ্ট।
১৩	ক	Service ID automation/অ্যাপের পরিচয়।
১৪	খ	Service-to-service authorization মানুষের credential ছাড়া কাজ করে।
১৫	খ	ফাঁস হলে revoke/rotate ও নিরীক্ষা প্রথম কাজ।
১৬	খ	Subnet network আলাদা ও নিয়ন্ত্রণ দেয়।
১৭	ক	Public Gateway/NAT egress দেয়, inbound খোলে না।
১৮	খ	Security Group stateful/instance; NACL stateless/subnet।
১৯	খ	স্থিতিশীল ব্যক্তিগত সংযোগে Direct Link ধরনের পথ।
২০	ক	/24-এ ২৫৬ ঠিকানা, কিছু সংরক্ষিত থাকে।
২১	খ	CPU/memory/storage workload অনুযায়ী।
২২	খ	আগে কারণ নির্ণয়, তারপর সাইজ বদল।

২৩	ক	Stateless-এ ডেটা instance-এ থাকে না।
২৪	খ	Patch-এর আগে backup ও rollback পরিকল্পনা।
২৫	খ	নির্দিষ্ট hardware/isolation দরকারে bare metal।
২৬	খ	Unstructured বড় ডেটায় object storage।
২৭	ক	৩ কপি, ২ মাধ্যম, ১ off-site।
২৮	খ	Public bucket থেকে সংবেদনশীল ডেটা ফাঁস হতে পারে।
২৯	খ	পরীক্ষা ছাড়া backup কাজ করবে তার নিশ্চয়তা নেই।
৩০	ক	Access frequency ও retention অনুযায়ী storage class।
৩১	ক	Managed ডেটাবেসে backup/patch/HA-এর অনেক কাজ provider করে।
৩২	খ	আগে query/index/connection দেখুন।
৩৩	খ	Pool শেষ হলে নতুন request অপেক্ষা/ব্যর্থ হয়।
৩৪	ক	Replication ও consistency latency বাড়ায়।
৩৫	খ	Manual UPDATE-এর আগে backup ও প্রভাব যাচাই।
৩৬	খ	Pod সবচেয়ে ছোট deployable একক।
৩৭	ক	Replica বাড়ালে availability বাড়ে।
৩৮	খ	Log/event, তারপর config/env/image।
৩৯	খ	Stateful-এ StatefulSet ও persistent volume দরকার।
৪০	ক	Service Pod-কে স্থিতিশীল ঠিকানা দেয়।
৪১	খ	OpenShift Route/Operator/workflow যোগ করে।
৪২	খ	Route OpenShift-এর; Ingress সাধারণ Kubernetes।
৪৩	ক	Operator জটিল অ্যাপের lifecycle স্বয়ংক্রিয় করে।
৪৪	খ	Multi-cluster-এ GitOps ও কেন্দ্রীয় নীতি।
৪৫	ক	Project/namespace resource আলাদা করে।
৪৬	ক	Serverless-এ সার্ভার পরিচালনা করতে হয় না।
৪৭	ক	Cold start-এ প্রথম request ধীর হতে পারে।
৪৮	ক	দীর্ঘ batch-এ timeout/memory/খরচের সীমা থাকতে পারে।
৪৯	খ	নতুন কাজে Code Engine-এর মতো বর্তমান প্ল্যাটফর্ম যাচাই করুন।
৫০	ক	Trigger ঘটনা ঘটলে কাজ শুরু করে।

৫১	খ	CI/CD পরীক্ষা/build/deploy স্বয়ংক্রিয় করে।
৫২	খ	Staging-এ functional/integration/rollback যাচাই।
৫৩	ক	Blue-green-এ দ্রুত switch ও rollback সহজ।
৫৪	খ	Canary ছোট অংশে পরীক্ষা করে বাড়ায়।
৫৫	খ	Secret secret manager-এ রাখুন, Git-এ নয়।
৫৬	খ	Terraform Infrastructure as Code।
৫৭	খ	State বাস্তব অবস্থার সঙ্গে মিল রাখে।
৫৮	খ	Remote backend-এ lock সহ state নিরাপদ।
৫৯	খ	Drift plan-এ দেখা যায়।
৬০	ক	Module পুনঃব্যবহার ও মান দেয়।

সংক্ষিপ্ত উত্তর — মূল পয়েন্ট

1. **Shared responsibility:** provider infrastructure/প্ল্যাটফর্মের নিরাপত্তা দেয়; গ্রাহক নিজের ডেটা, access ও কনফিগারেশনের দায়িত্বে থাকে।
2. **Resource group:** পরিবেশ আলাদা হলে access, billing ও ভুল deploy-এর ঝুঁকি কমে।
3. **Access Group:** ভূমিকা অনুযায়ী group-এ policy দিলে যোগ/বিয়োগ সহজ, ভুল কমে।
4. **Service ID:** অ্যাপ/automation-এর পরিচয়; ব্যক্তিগত key মানুষের, চাকরি বদলে ঝুঁকি বাড়ে।
5. **CIDR:** /16 বড় নেটওয়ার্ক; /24 তার ভেতরের ছোট subnet — পরিকল্পনায় overlap এড়ান।
6. **Security Group + NACL:** SG instance-স্তরে stateful; NACL subnet-স্তরে stateless — দুই স্তরে প্রতিরক্ষা।
7. **Stateless:** session/ডেটা বাইরে (ডেটাবেস/cache) থাকলে যেকোনো instance বদলানো যায়।
8. **Versioning/lifecycle:** ভুল মুছে ফেলা থেকে রক্ষা; পুরোনো ডেটা সস্তা class-এ সরানো/মুছে ফেলা যায়।
9. **Connection pool:** বেশি connection ডেটাবেসের resource শেষ করে; সীমা অ্যাপ ও ডেটাবেসের ক্ষমতা অনুযায়ী।
10. **Pod/Deployment/Service:** Pod চালায়, Deployment replica ও update পরিচালনা করে, Service স্থিতিশীল ঠিকানা দেয়।
11. **Probe:** liveness restart ঠিক করে; readiness traffic ঠিক করে — ভুল liveness বারবার restart ঘটায়।
12. **Route:** OpenShift-এ সহজ TLS/edge routing দেয়; Ingress সাধারণ Kubernetes-এ portable।
13. **Cold start:** প্রথম invocation-এ container তৈরি হয়; কমাতে warm instance, ছোট dependency, উপযুক্ত memory।
14. **Blue-green বনাম canary:** দ্রুত সম্পূর্ণ switch চাইলে blue-green; ঝুঁকি ধীরে যাচাই করতে canary।
15. **Drift:** `terraform plan`-এ ধরা পড়ে; যাচাই করে apply বা কনফিগারেশন সংশোধন করুন।
16. **Remote state/locking:** একসঙ্গে দুজন apply করলে state নষ্ট হতে পারে; lock তা রোধ করে।
17. **Ansible:** inventory কোথায়, playbook কী করবে — দুটো মিলে পুনরাবৃত্তিযোগ্য কনফিগারেশন।
18. **SLI/SLO/alert:** SLI পরিমাপ, SLO লক্ষ্য; alert তখনই যখন ব্যবহারকারী-প্রভাবিত লক্ষ্য ভাঙছে।

19. **p99:** ১% request খুব ধীর — downstream, GC, lock বা tail dependency দেখুন।
20. **RTO/RPO:** RTO ফিরতে কতক্ষণ; RPO কতটুকু ডেটা হারানো চলবে — উদাহরণ: RTO ৪ ঘণ্টা, RPO ১ ঘণ্টা।
21. **Least privilege:** container-এ non-root ও সীমিত capability; IAM-এ সীমিত role — দুটোতেই ন্যূনতম।
22. **Tagging:** tag ছাড়া কোন টিম/প্রজেক্টের খরচ কত তা ভাগ করা যায় না।
23. **Lock-in কমানো:** খোলা মান, portable API, ডেটা export পরিকল্পনা।
24. **Cross-cloud replication:** consistency, latency, খরচ ও conflict resolution একসঙ্গে কঠিন।
25. **Audit log:** কে কী বদলেছে তার প্রমাণ — compliance যাচাইয়ে ভিত্তি।
26. **GitOps drift:** console থেকে বদলালে Git ও বাস্তব অবস্থা আলাদা হয়; পরের deploy তা মুছে দিতে পারে।
27. **Blast radius:** সীমিত পরীক্ষায় production-এর বড় ক্ষতি এড়ানো যায়।
28. **Blameless postmortem:** দোষ নয়, ব্যবস্থার দুর্বলতা ঠিক করলে একই ভুল কমে।
29. **Capstone প্রমাণ:** deploy, rollback, monitoring, নিরাপত্তা, খরচ — পাঁচটিরই প্রমাণ।
30. **তিনটি শেখা:** নিজের উত্তর — অধ্যায়ের মূল ধারণা, একটি trade-off ও একটি operational অভ্যাস থাকতে হবে।

Volume 14 — Assessment & Answer Guide

উত্তর নির্দেশিকা ২ — MCQ ৬১–১২০, Scenario, Case Study ও Final Exam

MCQ উত্তর ৬১–১২০

প্রশ্ন	উত্তর	ব্যাখ্যা
৬১	খ	Ansible সার্ভার কনফিগারেশন ও deploy স্বয়ংক্রিয় করে।
৬২	খ	Idempotent: বারবার চালালেও একই কাঙ্ক্ষিত অবস্থা।
৬৩	ক	Group অনুযায়ী variable ও play আলাদা করা যায়।
৬৪	ক	Terraform resource বানায়; Ansible ভেতরে কনফিগার করে।
৬৫	ক	Vault secret এনক্রিপ্ট করে।
৬৬	খ	Tracing request-এর পথ দেখায়।
৬৭	খ	SLI/SLO-ভিত্তিক alert ব্যবহারকারী-প্রভাব দেখে।
৬৮	খ	Sensitive log redact করুন।
৬৯	খ	p95/p99 ও downstream দেখুন।
৭০	ক	Prometheus সংগ্রহ, Grafana প্রদর্শন।
৭১	খ	HA: অংশ ব্যর্থ হলেও সার্ভিস চলে।
৭২	ক	Multi-zone-এ একটি zone ব্যর্থ হলেও অন্যটি চলে।
৭৩	খ	RTO সময়; RPO ডেটা হারানোর সীমা।
৭৪	খ	পরীক্ষা ছাড়া runbook কাজ করবে কিনা জানা যায় না।
৭৫	ক	Load balancer traffic ভাগ করে।
৭৬	খ	Encryption at rest সংরক্ষিত ডেটার।
৭৭	ক	Customer-managed key-এ নিয়ন্ত্রণ বেশি।
৭৮	খ	Patch করা image-এ rebuild ও rescan করুন।
৭৯	খ	Zero trust: প্রতিটি access যাচাই।
৮০	খ	0.0.0.0/0 SSH সবার জন্য খোলা।
৮১	খ	FinOps স্বচ্ছতা, দায়িত্ব ও মূল্যের ভারসাম্য।
৮২	খ	আগে ব্যবহার যাচাই, তারপর সিদ্ধান্ত।

৮৩	ক	Tag দিয়ে খরচ ভাগ করে দেখা যায়।
৮৪	খ	কোন সার্ভিস, কখন থেকে, কী trigger — এটাই তদন্ত।
৮৫	খ	স্থিতিশীল baseline-এ committed ব্যবহার।
৮৬	খ	Multi-cloud: একাধিক provider একসঙ্গে।
৮৭	ক	জটিলতা, ভিন্ন মডেল ও egress খরচ প্রধান ঝুঁকি।
৮৮	খ	খোলা মান ও portable design lock-in কমায়।
৮৯	খ	Data residency যাচাই ছাড়া region বেছে নেবেন না।
৯০	ক	Hybrid: কিছু on-premises, কিছু cloud।
৯১	খ	Cross-cloud সংযোগ encrypted ও least-privilege হতে হবে।
৯২	খ	Bandwidth, latency, egress পথ ও ডেটার আকার দেখুন।
৯৩	খ	Federated identity ও কেন্দ্রীয় নীতি।
৯৪	খ	Failover-এ পরীক্ষিত runbook, replicated ডেটা ও DNS পরিকল্পনা লাগে।
৯৫	ক	API gateway সামঞ্জস্যপূর্ণ entry point দেয়।
৯৬	খ	Compliance ডেটার অবস্থান, access ও retention নির্ধারণ করে।
৯৭	ক	Audit log পরিবর্তনের প্রমাণ রাখে।
৯৮	খ	PII mask/redact ও retention ঠিক করুন।
৯৯	খ	Residency, replication ও backup পুনরায় নকশা করুন।
১০০	খ	Policy as Code: নিয়ম কোডে লিখে স্বয়ংক্রিয় যাচাই।
১০১	খ	GitOps: Git-ই সত্যের উৎস।
১০২	খ	পরিবর্তন Git commit/PR দিয়ে আসে।
১০৩	ক	GitOps টুল cluster-কে Git-এর সঙ্গে মেলায়।
১০৪	খ	Secret encrypted বা external manager-এ রাখুন।
১০৫	ক	Drift: Git ও বাস্তব অবস্থার পার্থক্য।
১০৬	খ	Chaos নিয়ন্ত্রিত ব্যর্থতায় resilience যাচাই করে।
১০৭	খ	Hypothesis, সীমা, rollback ও monitoring ছাড়া chaos নয়।
১০৮	খ	Latency, error, throughput ও resource একসঙ্গে।
১০৯	খ	p99 খারাপ মানে tail latency সমস্যা।
১১০	ক	Production-সদৃশ পরিবেশে ফল বাস্তব হয়।

১১১	খ	Incident-এ প্রথমে প্রভাব সীমিত ও যোগাযোগ।
১১২	খ	Blameless postmortem ব্যবস্থার দুর্বলতা ঠিক করে।
১১৩	খ	Runbook চাপের মধ্যে ধারাবাহিক পদক্ষেপ দেয়।
১১৪	খ	বারবার ঘটলে root cause ও স্থায়ী প্রতিকার।
১১৫	খ	On-call দায়িত্ব ভাগ করে ও ক্লান্তি কমায়।
১১৬	খ	Capstone-এ প্রথমে সমস্যা, ব্যবহারকারী ও সাফল্যের মাপকাঠি।
১১৭	খ	Multi-cloud যুক্তি প্রমাণ ও trade-off দিয়ে।
১১৮	খ	ব্যর্থতা থেকে শেখা ও বদল দেখানোই ভালো।
১১৯	খ	Production-প্রস্তুতির প্রমাণ: deploy, rollback, monitoring, নিরাপত্তা, খরচ।
১২০	খ	ছোট পরীক্ষা, নোট ও নিয়মিত অনুশীলন।

Scenario উত্তর — সংক্ষেপে

1. **সাইট timeout:** প্রথমে load balancer/health check ও Security Group/route — অ্যাপ log ঠিক থাকলে network পথই সন্দেহজনক। তারপর instance ও DNS।
2. **403:** প্রথমে IAM policy/access group ও resource group scope — login ঠিক, access-ই প্রশ্ন। তারপর role ও policy-র মেয়াদ।
3. **CrashLoopBackOff:** প্রথমে Pod log ও event — কেন crash হচ্ছে তা না জেনে restart অর্থহীন। তারপর env/config/image।
4. **ডেটাবেস ধীর:** প্রথমে slow query ও সক্রিয় session — CPU উপসর্গ, কারণ query হতে পারে। তারপর index ও connection।
5. **খরচ বৃদ্ধি:** প্রথমে billing breakdown — কোন সার্ভিস, কখন থেকে বেড়েছে। তারপর সেই resource-এর পরিবর্তন ইতিহাস।
6. **401:** প্রথমে key-র মেয়াদ/rotate/revoke ইতিহাস — গতকাল ঠিক ছিল, তাই পরিবর্তনই সন্দেহ। তারপর scope ও pipeline secret।
7. **DNS আংশিক:** প্রথমে DNS propagation ও record — কিছু ব্যবহারকারীর ক্ষেত্রে cache/propagation ভিন্ন হয়। তারপর TTL ও provider।
8. **Disk পূর্ণ:** প্রথমে বড় ফাইল/log ও disk ব্যবহার — কী ভরেছে তা না জেনে মুছবেন না। তারপর log rotation ও cleanup।
9. **502:** প্রথমে target health ও অ্যাপ port — load balancer পৌঁছাচ্ছে কিনা। তারপর অ্যাপ log ও timeout।
10. **State lock:** প্রথমে কে lock ধরেছে ও কতক্ষণ — চলমান apply থাকতে পারে। তারপর নিরাপদে unlock, জোর করে নয়।
11. **Object 404:** প্রথমে bucket/key নাম ও permission — নাম ভুল নাকি access ভুল। তারপর version ও lifecycle।
12. **Direct Link বন্ধ:** প্রথমে সংযোগের status ও সাম্প্রতিক পরিবর্তন — দুই প্রান্তের রুট। তারপর provider status।
13. **Memory লিক:** প্রথমে memory trend ও heap/profile — ধীরে বাড়া লিকের লক্ষণ। তারপর কোড/cache।

14. **ভুল সংস্করণ:** প্রথমে deploy করা image/tag ও running Pod-এর image মেলান — pipeline সফল মানেই সঠিক tag নয়। তারপর cache/CDN।
15. **Certificate expired:** প্রথমে certificate-এর মেয়াদ ও কোন ডোমেইনে — renewal/auto-renewal ব্যর্থ কিনা। তারপর DNS/validation।
16. **Cross-cloud latency:** প্রথমে network পথ, latency ও egress — অ্যাপের আগে সংযোগ। তারপর payload ও API-র নিজের latency।
17. **Restore ব্যর্থ:** প্রথমে backup-এর integrity, version ও storage access — restore-এর আগেই backup যাচাই। তারপর runbook ধাপ।
18. **Alert ঝড়:** প্রথমে মূল কারণের alert ও correlation — একটি ব্যর্থতা অনেক alert তৈরি করে। তারপর alert deduplication।
19. **ImagePullBackOff:** প্রথমে image নাম/tag ও registry access — টাইপো বা permission সাধারণ কারণ। তারপর secret ও network।
20. **Region বিভ্রাট:** প্রথমে provider status ও blast radius — এক region-এর সমস্যা অন্য region-এ ছড়াবেন না। তারপর failover পরিকল্পনা।

Case Study বিশ্লেষণ — মূল পয়েন্ট

1. **Security Group:** পরিবর্তন ও deploy আলাদা করুন; change review, staging যাচাই ও দ্রুত rollback।
2. **API key ফাঁস:** revoke, rotate, audit; pre-commit scan ও secret manager বাধ্যতামূলক করুন।
3. **Migration:** backup যাচাই, staging-এ পরীক্ষা, reversible migration, RPO অনুযায়ী backup।
4. **Zone down:** multi-zone, health check, স্বয়ংক্রিয় failover ও নিয়মিত মহড়া।
5. **খরচ:** দৈনিক alert, budget, rate limit ও anomaly detection; loop-এ guardrail।
6. **Memory লিক:** restart নয়, profiling ও fix; limit/alert ও trend monitoring।
7. **ভুল region:** Policy as Code, backup নীতির review ও residency checklist।
8. **ভুল image tag:** immutable tag, deploy verification ও canary metric gate।
9. **DNS/certificate:** পরিবর্তনের checklist, certificate মেয়াদ alert, rollback।
10. **Failover ব্যর্থ:** পরীক্ষা ছাড়া পরিকল্পনা নয়; নিয়মিত মহড়া, ডেটা freshness ও DNS TTL পরিকল্পনা।

Final Exam — Marking Guidance

- **MCQ:** প্রতিটি সঠিক ১, ভুল ০; অনুমান নয়, ব্যাখ্যা বোঝা গুরুত্বপূর্ণ।
- **সংক্ষিপ্ত উত্তর (৩ নম্বর):** সঠিক ধারণা ১, উদাহরণ/প্রেক্ষাপট ১, trade-off বা সতর্কতা ১।
- **Scenario (৫ নম্বর):** প্রথম চেক ও কারণ ২, পরের ধাপ ২, প্রতিরোধমূলক ব্যবস্থা ১।
- **Design (১৫ নম্বর):** architecture সম্পূর্ণতা ৫, নিরাপত্তা ৩, backup/HA ৩, monitoring/খরচ ২, trade-off স্পষ্টতা ২।
- **Incident (২০ নম্বর):** বিভাগ ঙ-এর ভাগ অনুযায়ী — timeline/মূল কারণ ৫, তাৎক্ষণিক প্রতিকার ৫, স্থায়ী ব্যবস্থা ১০।

Capstone Grading Rubric — সংক্ষেপ

মাত্রা	Excellent	Good	Needs work
Architecture	প্রয়োজন অনুযায়ী, trade-offসহ, multi-zone	মূলত ঠিক, কিছু ফাঁক	অসম্পূর্ণ বা যুক্তিহীন
Automation	IaC + CI/CD + GitOps, পরীক্ষিত	আংশিক automation	Manual, পুনরাবৃত্তিযোগ্য নয়
Security	Least privilege, secret, encryption, scan	মূল নিয়ন্ত্রণ আছে	বড় ফাঁক (public secret/port)
Operations	Monitoring, alert, runbook, DR পরীক্ষিত	মৌলিক monitoring	কোনো observability নেই
Cost	Tag, budget, right-sizing প্রমাণ	খরচের ধারণা আছে	খরচ বিবেচনা নেই
Presentation	স্পষ্ট, প্রমাণসহ, ব্যর্থতা থেকে শেখা	বোধগম্য	অস্পষ্ট, প্রমাণহীন